1. Problem Title & Source

Title: **Count Elements Greater Than All Previous Elements**

Source: **TCS NQT Previous Coding Questions**

2. Problem Statement (Short Summary)

You are given an integer array Arr of size N.

Your task is to **count how many elements are strictly greater than all the elements that appear before them** in the array.

Important rule:

The first element is always counted.

Because there are **no previous elements to compare with**.

3. Examples

Example 1

Input

5

7 4 8 2 9

Output

3

Explanation

Elements satisfying the condition:

7 → first element

8 → greater than 7 and 4

9 → greater than all previous elements

Total count = **3**

Example 2

Input

6

3 5 2 7 6 9

Output

4

Explanation

3 → first element

5 → greater than 3

7 → greater than 5

9 → greater than 7

Count = **4**

4. Constraints

$1 \le N \le 20$

$1 \le Arr[i] \le 10000$

Observation

The array size is small, but we should still aim for the **optimal solution**.

5. Core Concept (Pattern)

Prefix Maximum

Key idea:

Track the **maximum value seen so far** while traversing the array.

6. Thought Process

Brute Force Idea

For each element:

Check all previous elements.

Example

```
for i
   check j < i
```

Time complexity

$O(n^2)$

Optimized Idea

Maintain a variable:

maxSoFar

Steps:

1. Initialize

maxSoFar = arr[0]

count = 1

2. Traverse array

If

arr[i] > maxSoFar

Then

count++

update maxSoFar

This makes the solution **O(n)**.

7. Visualization

Example

[7,4,8,2,9]

Traversal

max = 7

count = 1

Steps

4 → not greater

8 → greater → count=2

2 → not greater

9 → greater → count=3

Final result

3

8. Pseudocode

```
read n
read array

maxSoFar = arr[0]
count = 1

for i from 1 to n-1

   if arr[i] > maxSoFar
      count++
      maxSoFar = arr[i]

print count
```

9. Code Implementation

Python

```
n = int(input())
arr = list(map(int,input().split()))

max_so_far = arr[0]
count = 1
```

```
for i in range(1,n):

    if arr[i] > max_so_far:
        count += 1
        max_so_far = arr[i]

print(count)
```

Java

```
import java.util.*;

class Main {

    public static void main(String[] args){

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[] arr = new int[n];

        for(int i=0;i<n;i++)
            arr[i] = sc.nextInt();

        int maxSoFar = arr[0];
        int count = 1;

        for(int i=1;i<n;i++){

            if(arr[i] > maxSoFar){

                count++;
                maxSoFar = arr[i];
            }
        }
```

```
        System.out.println(count);
    }
}
```

10. Time & Space Complexity

Time Complexity

O(n)

Single traversal of array.

Space Complexity

O(1)

Only a few variables used.

11. Common Mistakes

1. Forgetting to count the first element
2. Using >= instead of >
3. Not updating maxSoFar
4. Starting loop from index 0

12. Detailed Dry Run

Input

[3,5,2,7,6,9]

Index	Value	maxSoFar	Count
0	3	3	1
1	5	5	2
2	2	5	2
3	7	7	3
4	6	7	3
5	9	9	4

Output

4

13. Real-Life Applications

Prefix maximum pattern appears in:

- Stock price analysis
- Trend detection

- Performance tracking
- Streaming data analysis

Example

Detect new highest stock price

14. Common Traps

1. Incorrect initialization
2. Off-by-one errors
3. Using nested loops unnecessarily
4. Confusing with prefix sum

15. Related Problems

Prefix-based problems:

LC 121 Best Time to Buy and Sell Stock

LC 42 Trapping Rain Water

LC 53 Maximum Subarray

16. Alternate Approaches

Approach	Complexity	Notes
Brute Force	$O(n^2)$	compare with all previous
Prefix Maximum	$O(n)$	optimal

17. Why This Solution Works

At each step we only need to know the **largest element encountered so far**.

If the current element exceeds this maximum, then it must be **greater than all previous elements**.

Thus we avoid unnecessary comparisons.

18. Variations / Follow-Up Questions

Possible interview variations:

1. Count elements **greater than next elements**
2. Find **maximum prefix values**
3. Track **record-breaking days**
4. Longest increasing prefix